Xebia

Modernizing Legacy Code with AI

A Practical Approach to Brownfield Projects

Eric Greene · Xebia

eric.greene@xebia.com

Agenda

- Why brownfield is harder for AI
- Using AI to understand unfamiliar code
- Building a safety net before refactoring
- Strangler fig, targeted refactors, and dependency upgrades
- Translating, untangling, and rewriting while preserving behavior
- Knowing when to trust, intervene, or leave alone

Why brownfield is harder for AI

Missing context

The reasons left with the people

Docs and comments go stale

Implicit constraints

Other systems need exact output

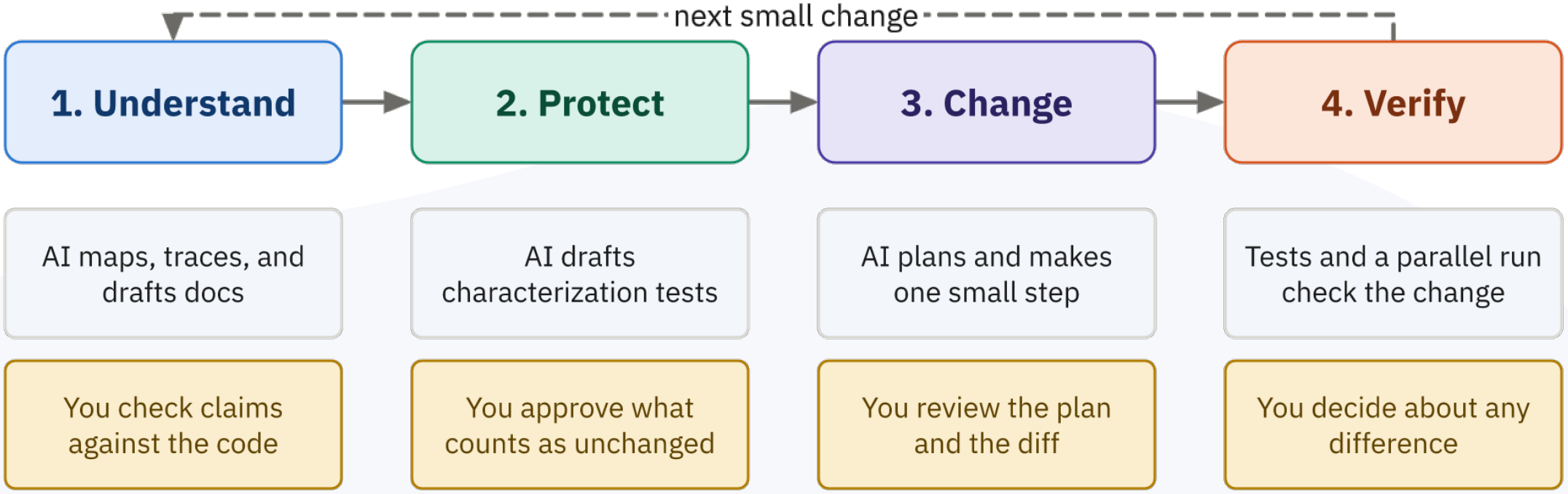
Quirks already on every invoice

Accumulated tech debt

God functions and global state

Old dependencies, few tests

A loop for brownfield work



Using AI to understand unfamiliar code

- Run it first, and read the real output
- Scope the agent to code, not old docs
- Map the architecture, then trace one real calculation
- Ask for `file:line` evidence for every claim
- Ask what the code can't tell you
- Generate docs, then review them like code

Verify the map, then keep it

Check the claims

A mechanical map: imports, callers, dead code

Open the lines the agent cites

Run the code to confirm a rule

Capture the context

Reviewed architecture notes for people

Custom instructions for every AI session

Open questions with named owners

Demo 1: Read before you change

demos/01-understand-legacy · about 17 minutes

Characterization tests

- Pin what the code does, not what it should
- Expected values come from running the real code
- Pin the quirks too, and label them
- Find a seam to test without changing code
- Bugs become open questions, not fixes

Pinning a quirk, on purpose

```
# tests/test_util.py
def test_round2_rounds_down_a_half_cent_produced_by_float_math():
    # 6.95 + 0.95 * 8.5 is 15.024999... in floats,
    # so customers were billed 15.02, not 15.03.
    assert util.round2(rates.base_charge("ECO", 4, 9.5)) == 15.02
```

Two layers of safety net

Golden master

A whole month's output, compared byte for byte

Fast to build, but only says something changed

A person reviews it before it's approved

Focused characterization tests

One rule per test, named after the rule

A failure names the rule that broke

Coverage shows the gaps, not the safety

Demo 2: Build the safety net

demos/02-safety-net · about 18 minutes

Incremental modernization patterns

Targeted refactor

Change structure, never behavior

One step, tests before and after

Dependency upgrade

Warnings show what will break

One package, fix by cause

Strangler fig

Grow new code behind a seam

Move one slice at a time

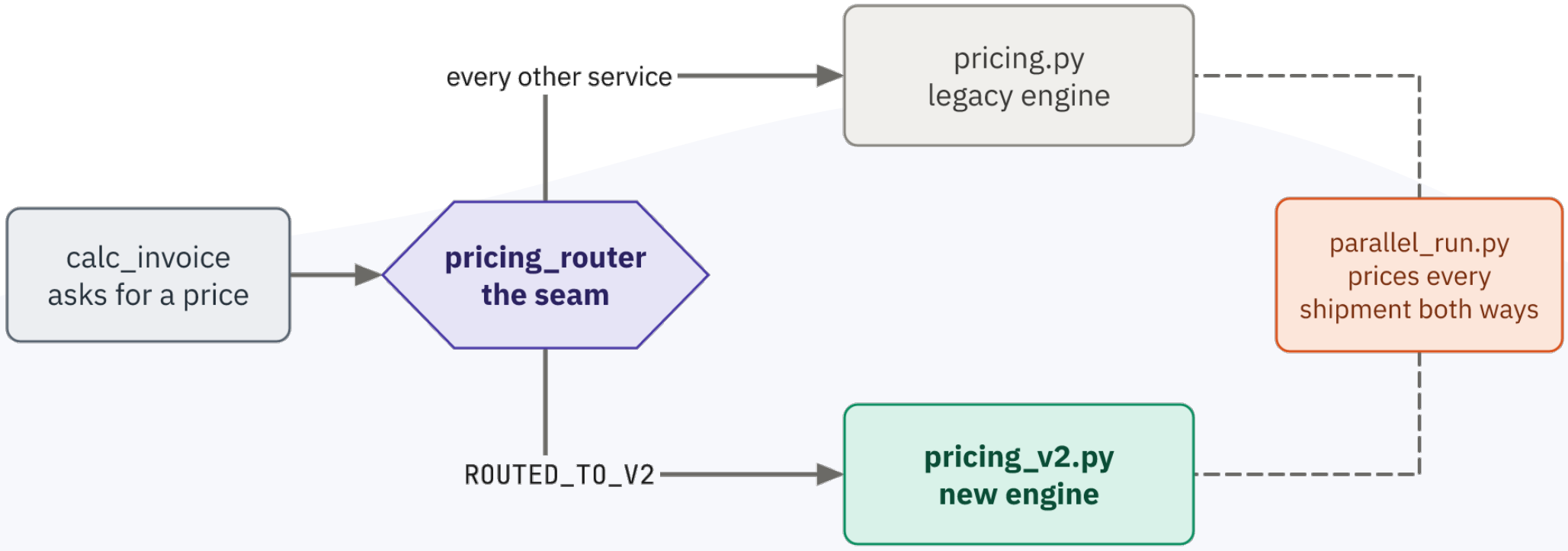
Refactors and upgrades with an agent

- Plan first, and read the plan for behavior changes
- Prompt files make the safe workflow repeatable
- The agent runs the tests; you review each diff
- Never let the agent edit a test to pass
- Group failures by cause before fixing anything
- Finish with warnings treated as errors

Demo 3: Modernize in small steps

demos/03-small-steps · about 17 minutes

The strangler fig



One service at a time: ECO → STD → EXP → OVN → delete pricing.py

Translate, untangle, or rewrite

Translate

New language, same results

Recorded runs become the test

Untangle

Find seams, extract pure functions

Same arithmetic, same order

Rewrite

New code beside the old code

A parallel run on real data decides

Knowing the limits

Trust the agent

Reading, mapping, drafting tests

Changes under a strong safety net

Intervene

Even a one-cent change

Edits to tests or external contracts

Leave it alone

Works, stable, has a contract

Guard it with a golden master

Demo 4: Strangle, rewrite, and know when to stop

demos/04-strangler-fig · about 18 minutes

Next steps

- Clone the repo and run the four demos
- Pick one legacy module and map it first
- Put a golden master around it before changing it
- Write your team's rules into custom instructions
- Adapt the repo's modernization checklist for your team



Thank you

Eric Greene · Xebia

eric.greene@xebia.com

xebia.com